

软件工程

第4章 软件设计

4.3 体系结构模式(第1部分: MVC模式与发布—订阅模式)

谭庆平 教授

国防科技大学 计算机学院

2015年3月

内容提要

1.

- MVC模式

2.

- 发布—订阅模式

3.

- 微内核模式

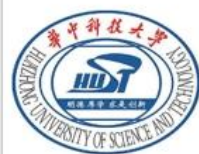
4.

- 反射模式

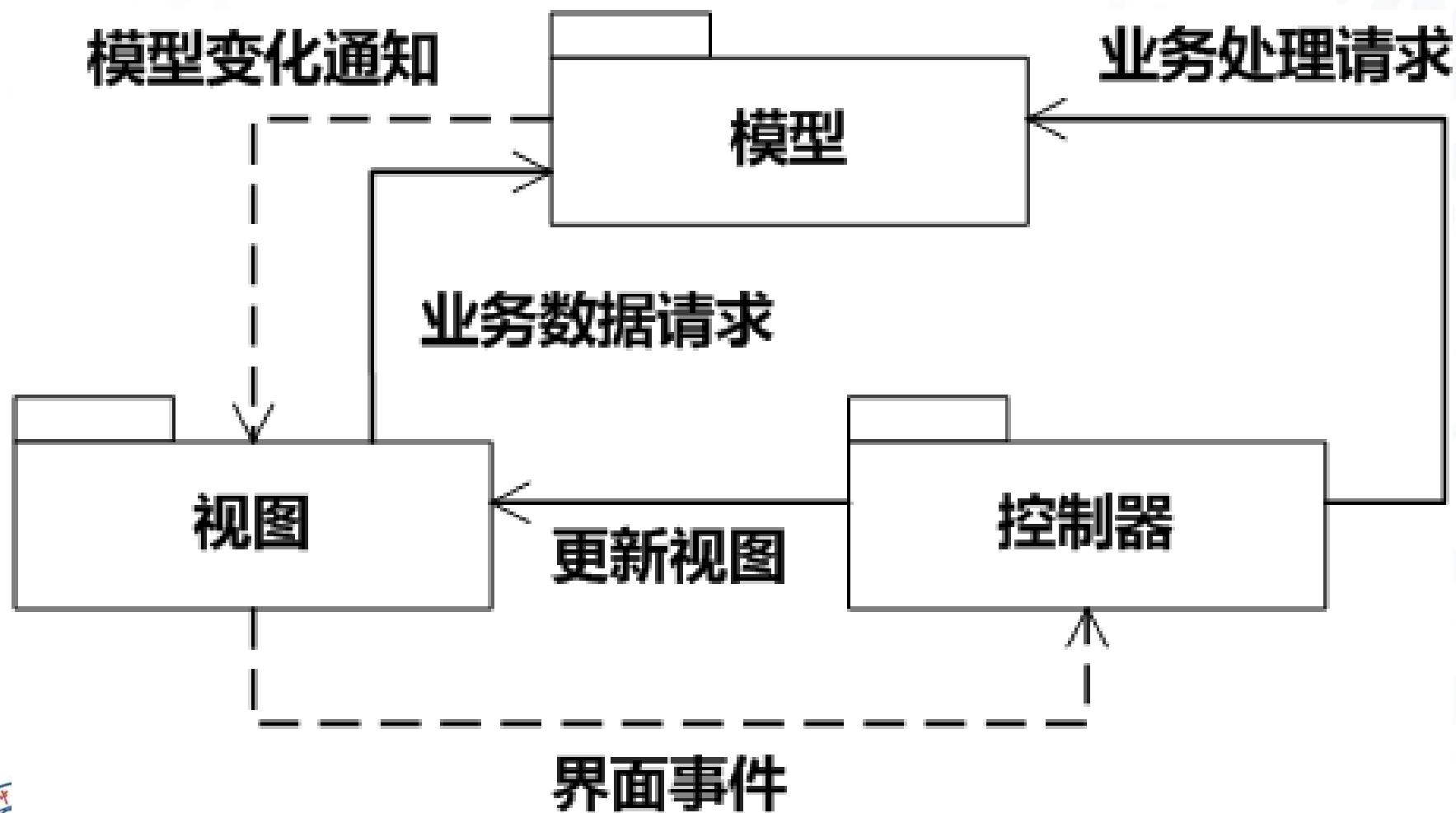
MVC模式的适用场景和问题

□ 要求目标软件系统具有灵活的人机交互接口，例如

- ◆ 相同的业务数据在不同窗口呈现为不同的表示形式；
- ◆ 对用户界面的修改尽量不影响系统内部的业务处理逻辑；
- ◆ 支持不同的界面风格，甚至不同类型的终端设备；等

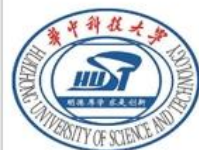


MVC模式示意图



MVC模式示意图的解释性说明(1)

- 将系统划分为三类模块：模型（M）、视图（V）和控制器（C）
- 模型的职责：
 - ◆ 存储业务数据并提供业务逻辑处理功能
 - ◆ 一旦业务数据有变化，通知视图



MVC模式示意图的解释性说明(2)

□ 视图的职责:

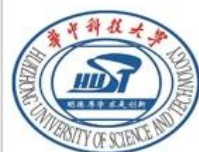
◆ 呈现模型中的数据

☑ 单个模型可对应多个视图，以呈现不同的数据表现形式

◆ 在接获模型变化通知后从模型中获取新数据并更新视图

◆ 创建 (或绑定已存在的)控制器，初始化控制器

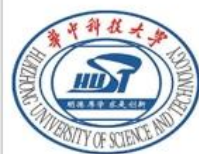
◆ 接受界面输入，转换为内部事件传递给控制器



MVC模式示意图的解释性说明(3)

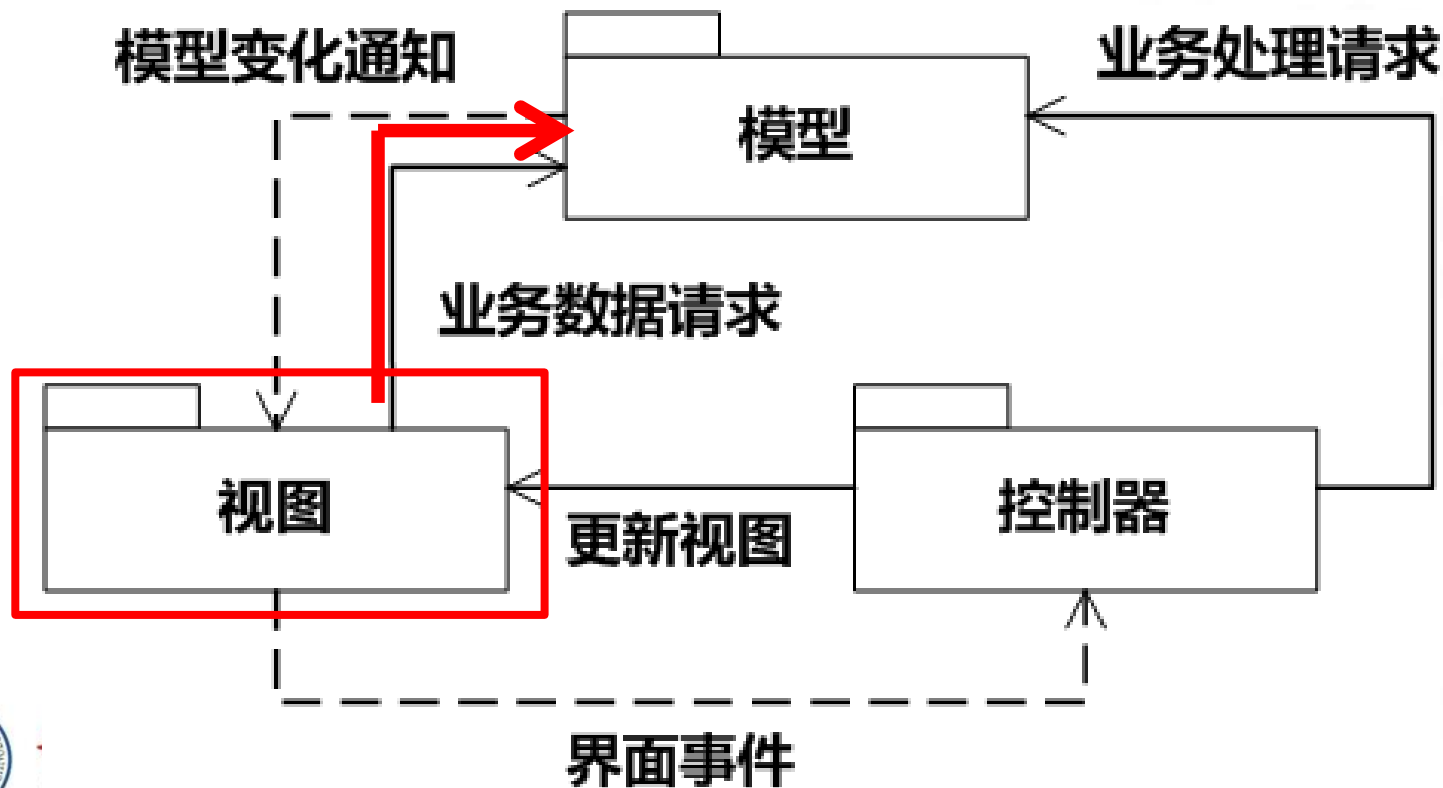
□ 控制器的职责：

- ◆ 将来自视图的界面事件转换为对模型的业务逻辑处理请求，或者对视图的展示请求
- ◆ 在接获模型的业务逻辑处理结果后负责选择适当的视图(并传递适当的数据给选定的视图)作为软件系统对界面动作的响应



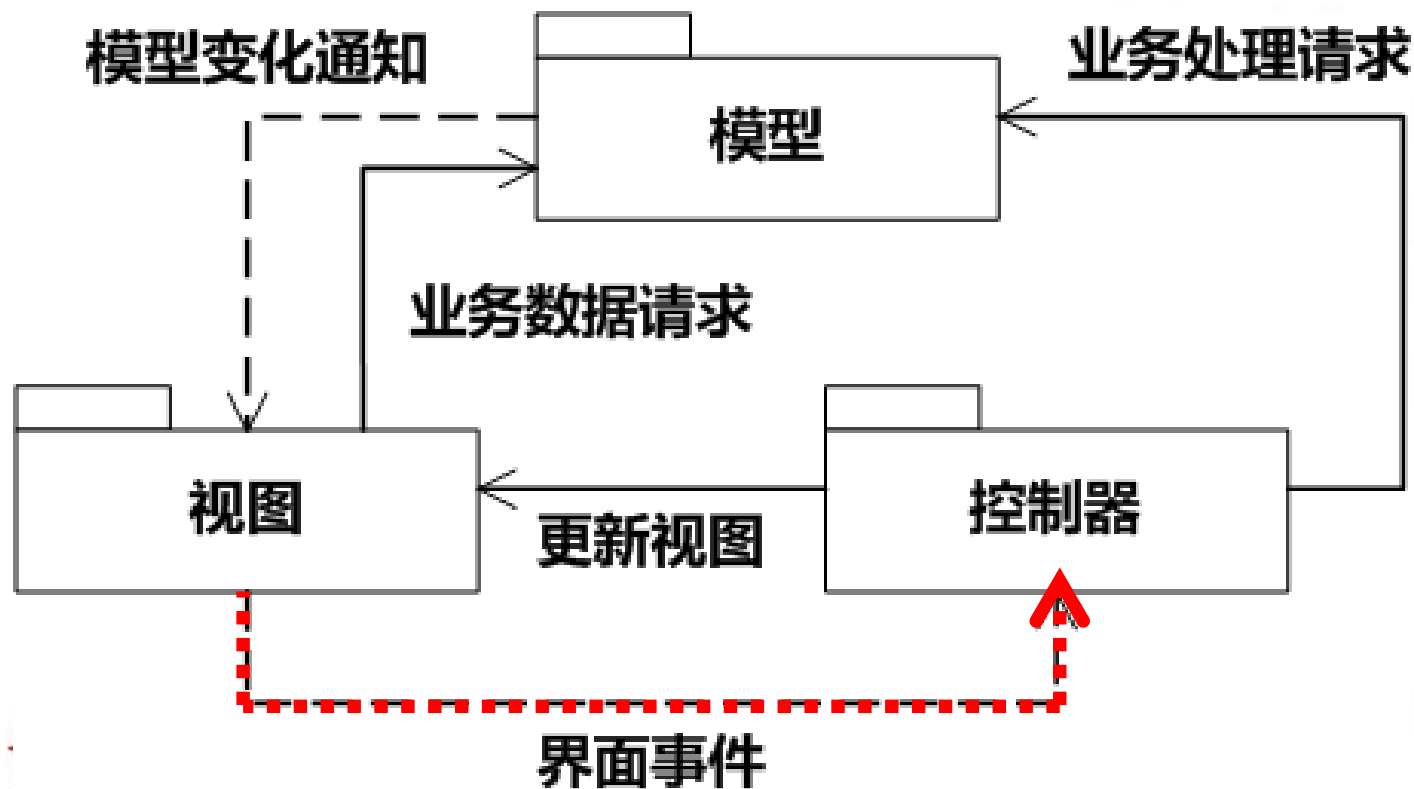
MVC的典型运作流程(1)

1. 创建视图，视图从模型中获取数据并呈现至界面



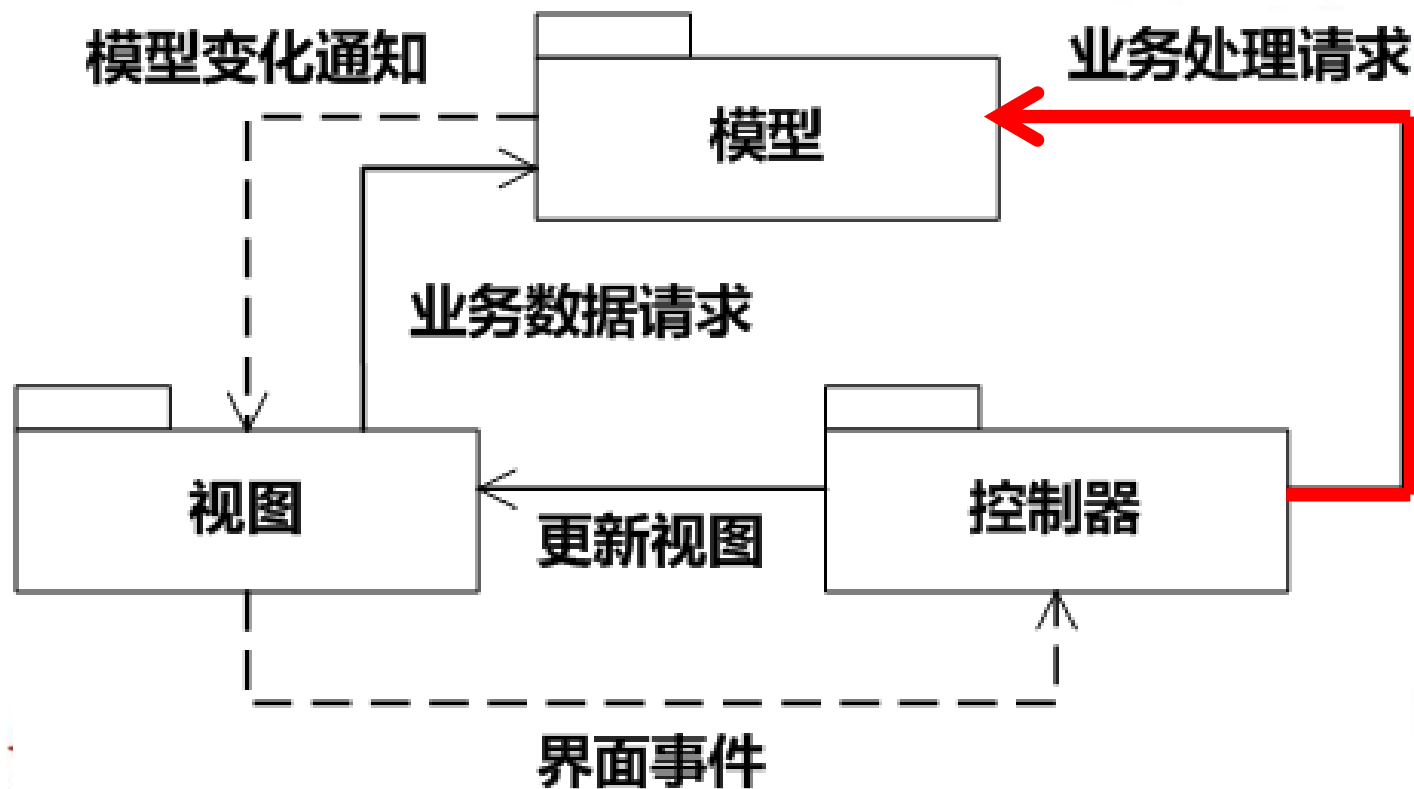
MVC的典型运作流程(2)

2. 视图接受界面动作，将其转换为内部事件传递给控制器



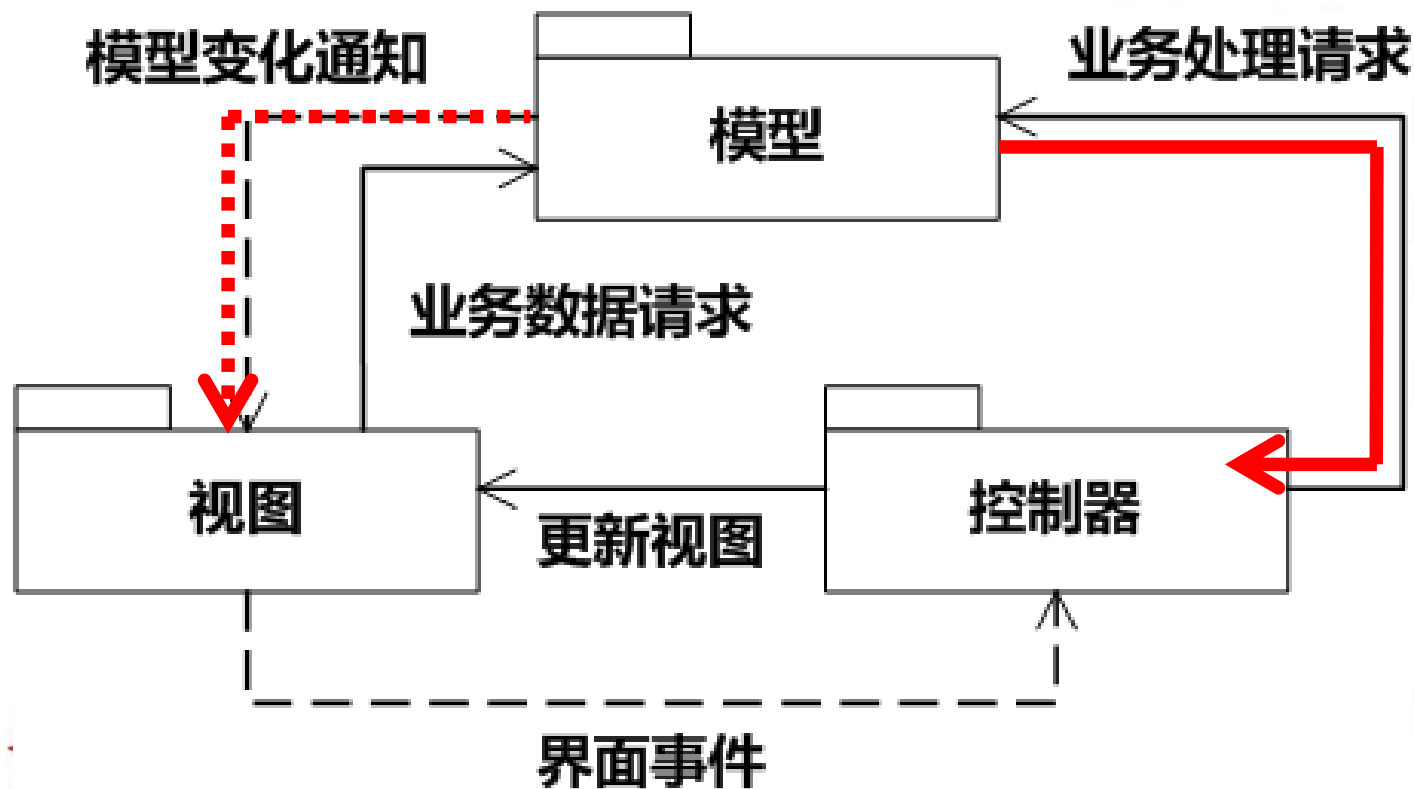
MVC的典型运作流程(3)

3. 控制器将界面事件转换为对模型的业务逻辑处理功能的调用



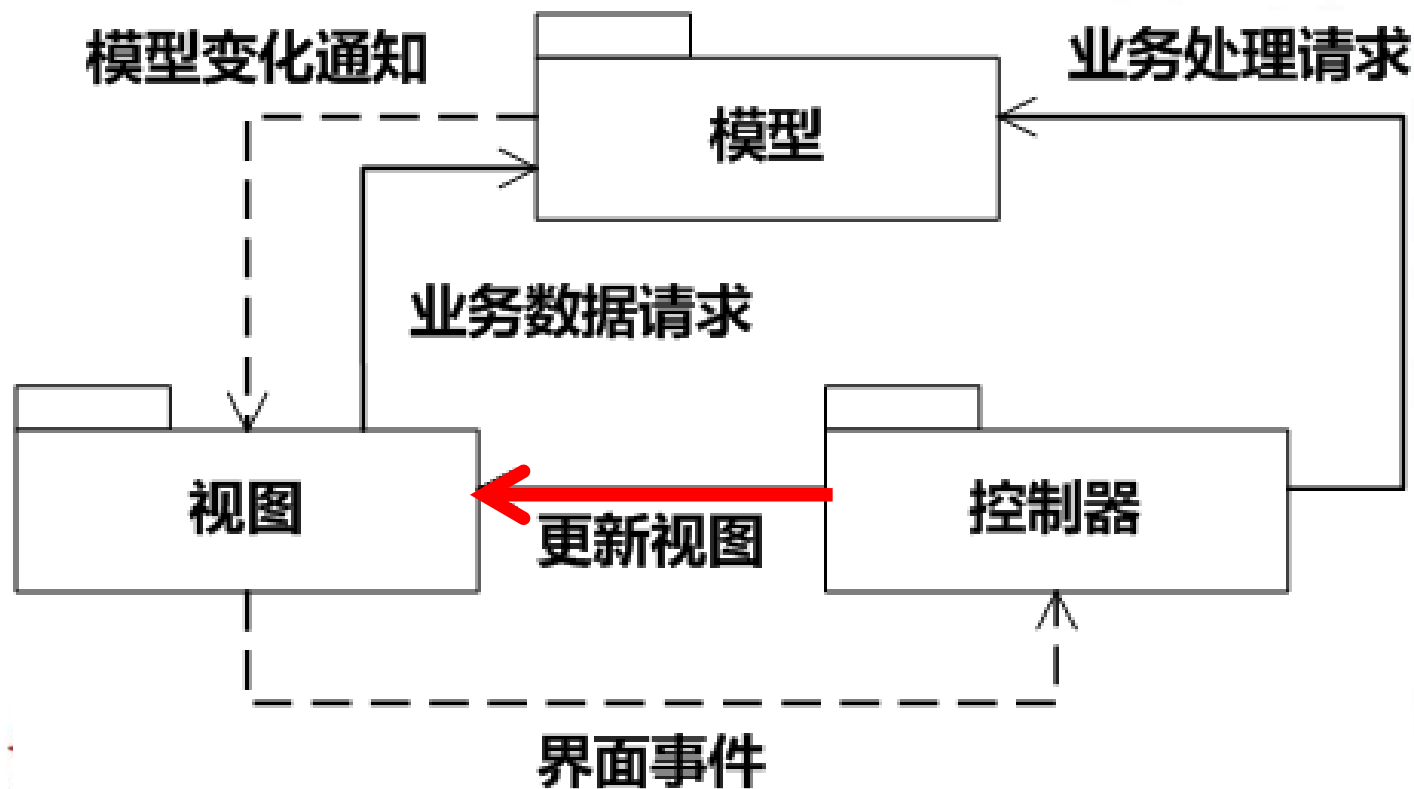
MVC的典型运作流程(4)

4. 模型进行业务逻辑处理，完成后将结果回送给控制器，必要时将变化事件通知给所有现行视图



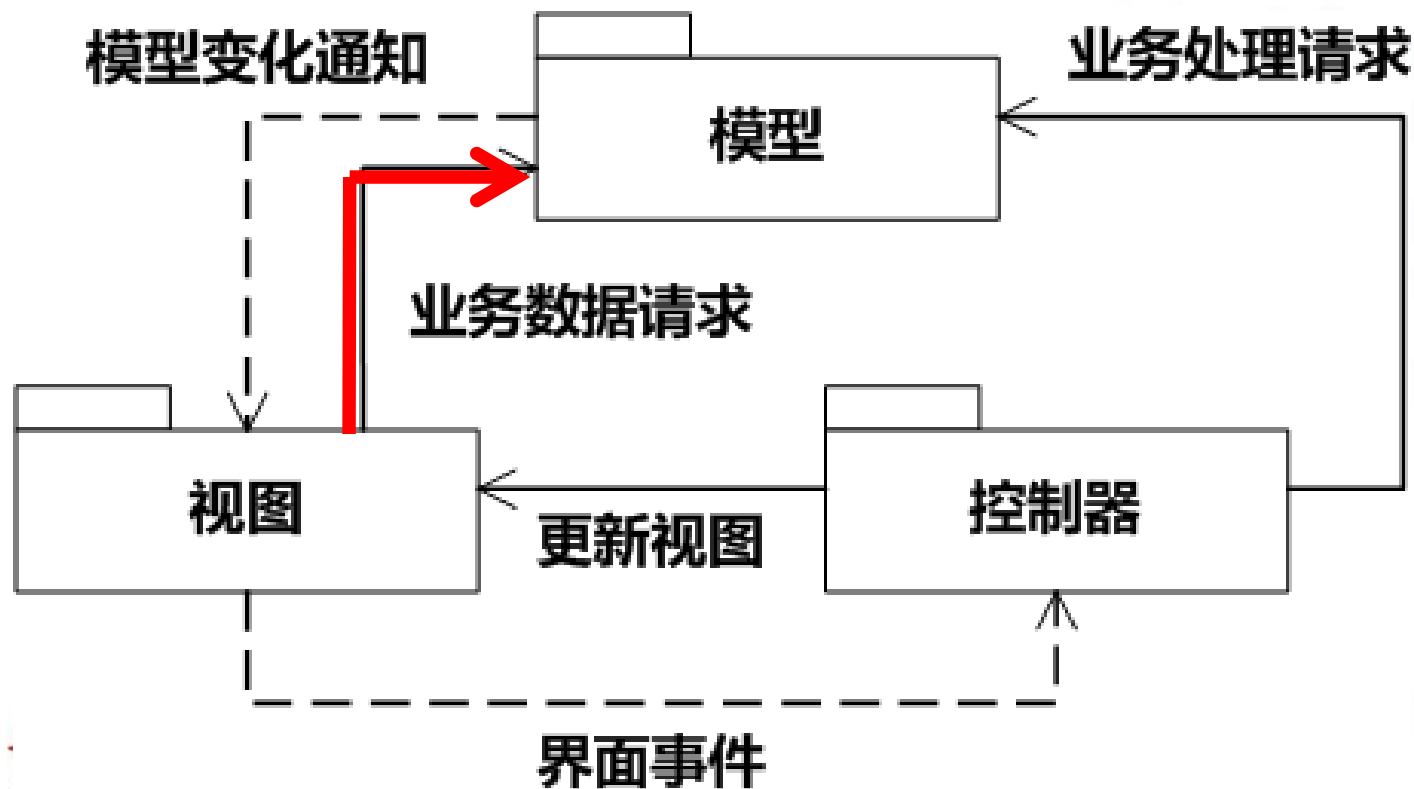
MVC的典型运作流程(5)

5. 控制器根据模型的处理结果创建新的视图、选择其他视图或维持原有视图，传递处理结果给视图



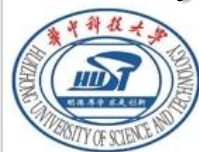
MVC的典型运作流程(6)

6. 所有视图在接获来自模型的变化通知后，向模型查询新的数据，并据此更新视图



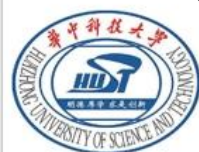
MVC的正面效应

1. 将模型与视图分离以支持同一模型的多种呈现，界面的式样和观感可动态切换而不影响模型
2. 将视图与控制器分离以支持在软件运行时根据业务逻辑处理结果选取最适当的视图
3. 将模型与控制器分离以支持从界面动作到业务处理行为之间的映射的**可配置性**
4. 模型与视图之间的**变更—通知机制**确保视图与业务数据的适时同步

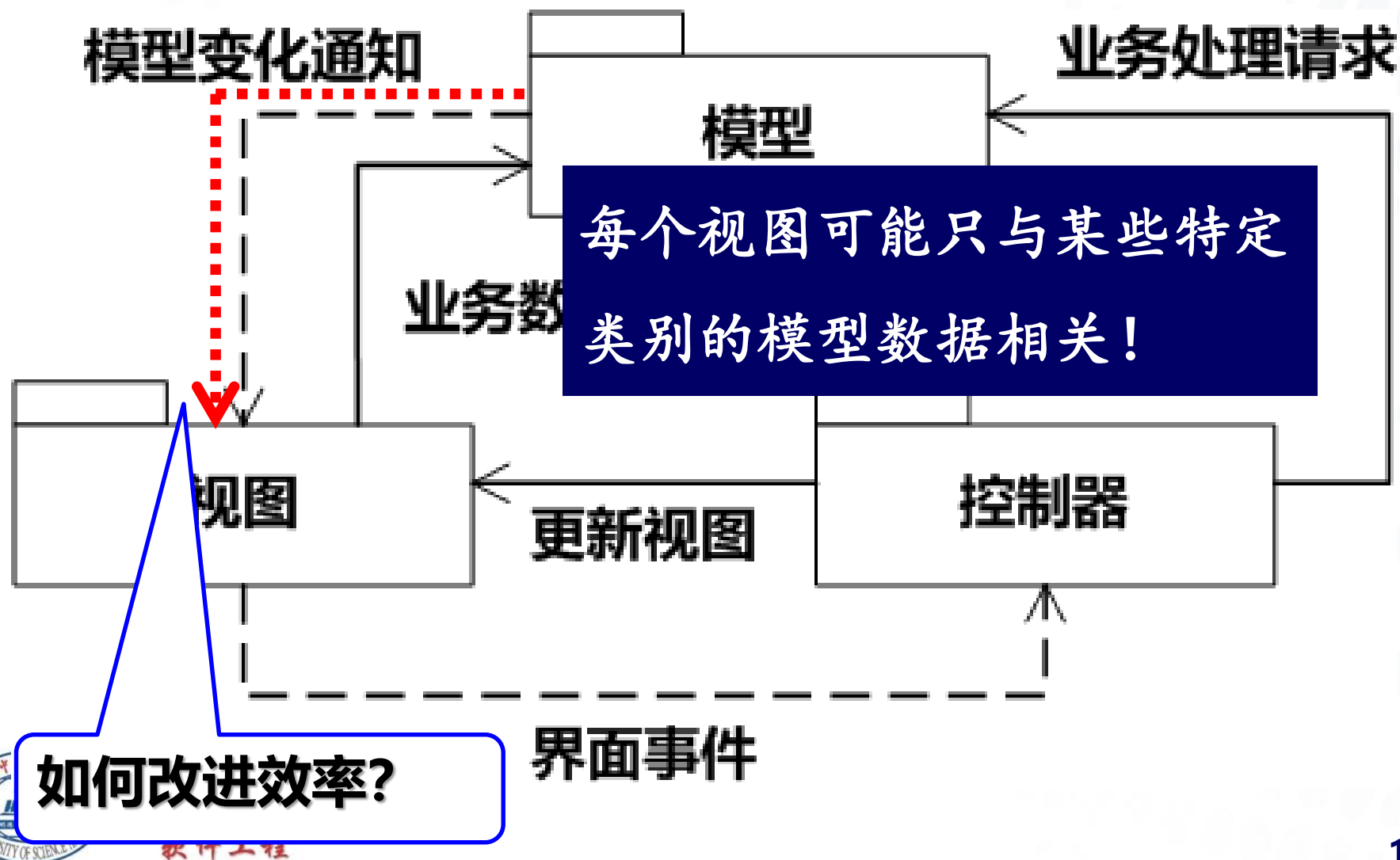


MVC的负面效应

1. 由于模型、视图、控制器三者之间的分离而导致的**开发复杂性**和**运行性能开销**
2. 视图与控制器耦合度较大
3. 视图从模型多次获取数据可能导致低效
 - ☑ 精心设计模型的数据访问接口
 - ☑ 视图cache从模型中获取的数据
4. 来自模型的频繁的变化通知可能导致低效

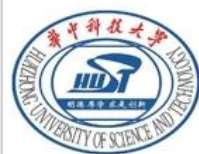


再次审视MVC模式



尝试性的解决方案

□ 是否存在一种机制，确保模型的数据变化通知 仅发给 有需要（感兴趣）的视图？



内容提要

1.

• MVC模式

2.

• 发布—订阅模式

3.

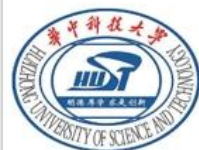
• 微内核模式

4.

• 反射模式

发布—订阅模式的适用场景和问题(1)

- 若干组件依赖于某个组件的内部状态，即，当该组件的内部状态发生变化时，其他的依赖组件必须及时得到通知
 - ◆ 因为低效，不能让依赖组件 **轮询** 被依赖组件
 - ◆ 因为只有部分依赖组件对特定类型的状态变化感兴趣，不能让 被依赖组件 向 **所有**依赖组件 传递状态变化通告



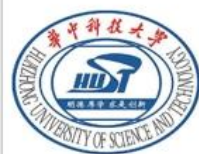
发布—订阅模式的适用场景和问题(2)

□ 若干组件依赖于某个组件的内部状态，即，当该组件的内部状态发生变化时，其他的依赖组件必须及时得到通知

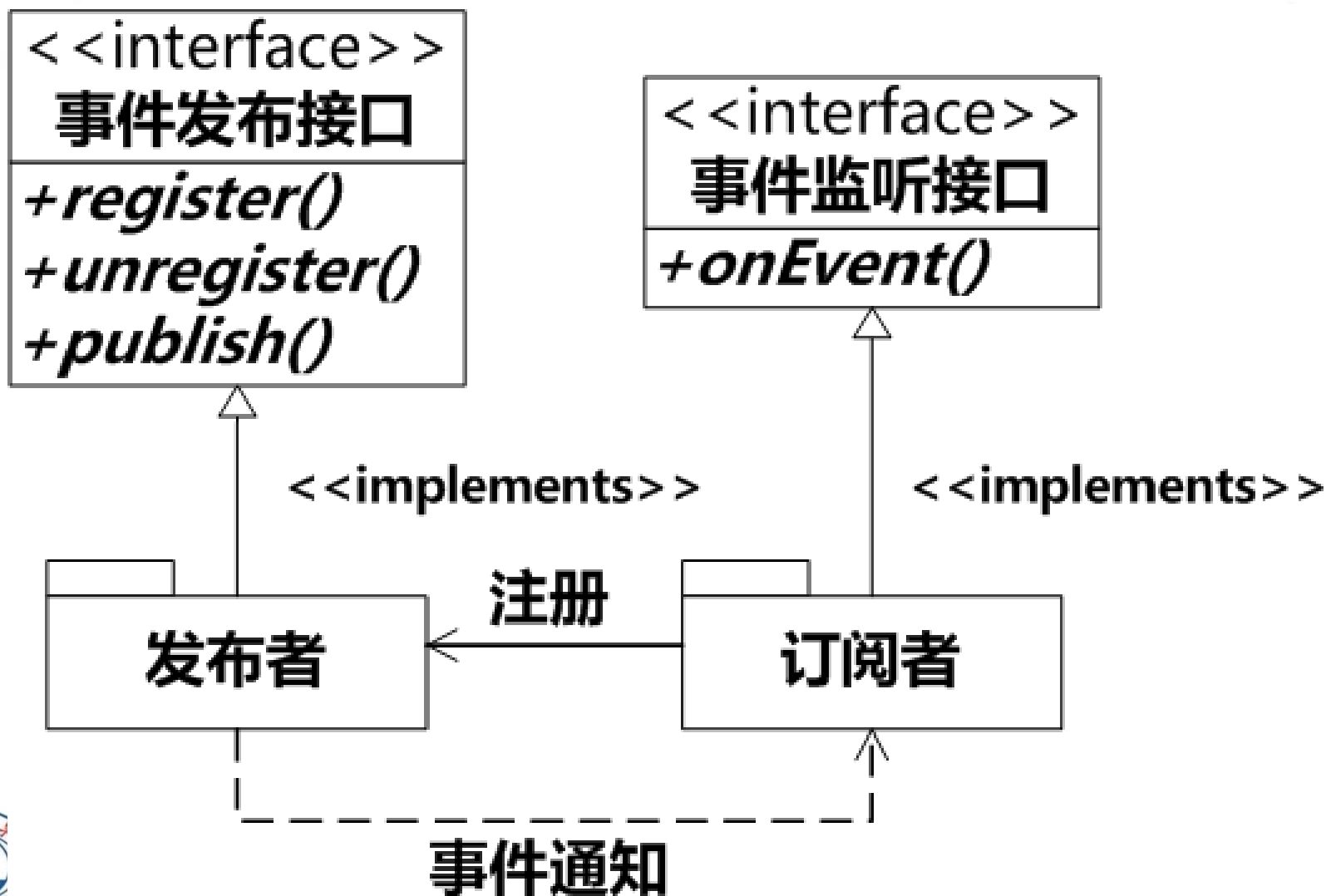
◆ 依赖组件的数目和类别不能事先确定，甚至它们可动态变化

☑ 必须设法降低 依赖组件 与 被依赖组件 之间的耦合度

怎么办?



发布—订阅模式示意图



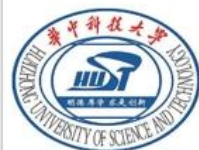
发布—订阅模式示意图的解释性说明

□ 发布者的职责：

- ◆ 接受订阅者的注册 及 撤销注册
- ◆ 一旦状态有变化，通知已注册的订阅者

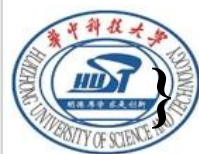
□ 订阅者的职责：

- ◆ 向发布者注册（不再感兴趣时撤销注册）
- ◆ 当状态变化事件到达时，处理该事件



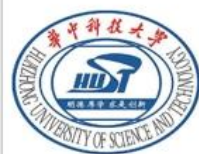
发布—订阅模式示例性实现(1)

```
public class Publisher implements IPublisher {  
    private List<EventListener> m_listeners;  
  
    public void register(EventListener listener) {  
        // .....  
        m_listeners.add(listener);  
    }  
  
    public void unregister(EventListener listener) {  
        // .....  
        m_listeners.remove(listener);  
    }  
}
```



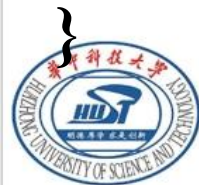
发布—订阅模式示例性实现(2)

```
public class Publisher implements IPublisher {  
// .....  
  
public void publish(Event event) {  
    if (m_listeners != null) {  
        for (EventListener listener : m_listeners) {  
            listener.onEvent();  
        }  
    }  
}
```



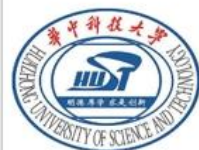
发布—订阅模式示例性实现(3)

```
public class Publisher implements IPublisher {  
    // .....  
  
    public void someFunc() {  
        // .....  
        Event event = new Event(/* ..... */);  
        publish(event);  
        // .....  
    }  
}
```



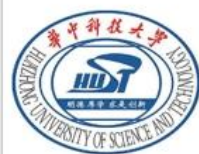
发布—订阅模式示例性实现(4)

```
public class Subscriber implements  
    IEventListener {  
  
    // .....  
  
    public void onEvent(Event event) {  
        // get data from event  
        // .....  
    }  
}
```



发布—订阅模式的正面效应

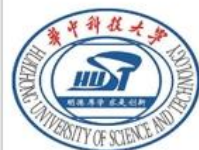
- 从无目的的事件广播转变为定向事件通告，避免无效传播，降低性能开销
- 降低发布者、订阅者之间的耦合度



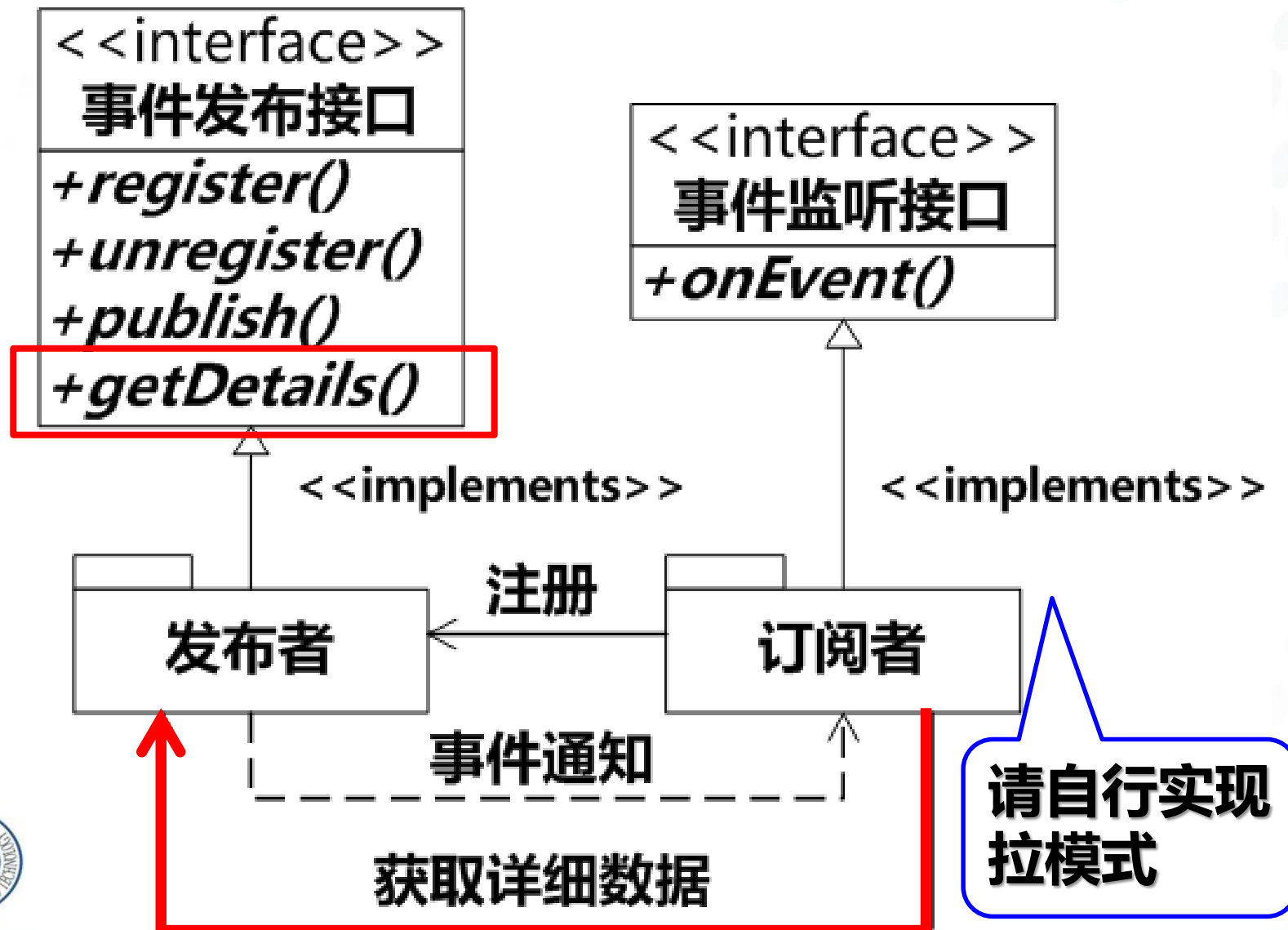
发布—订阅模式的负面效应

- 如果从发布者发向订阅者的数据量太大，仍可能存在浪费

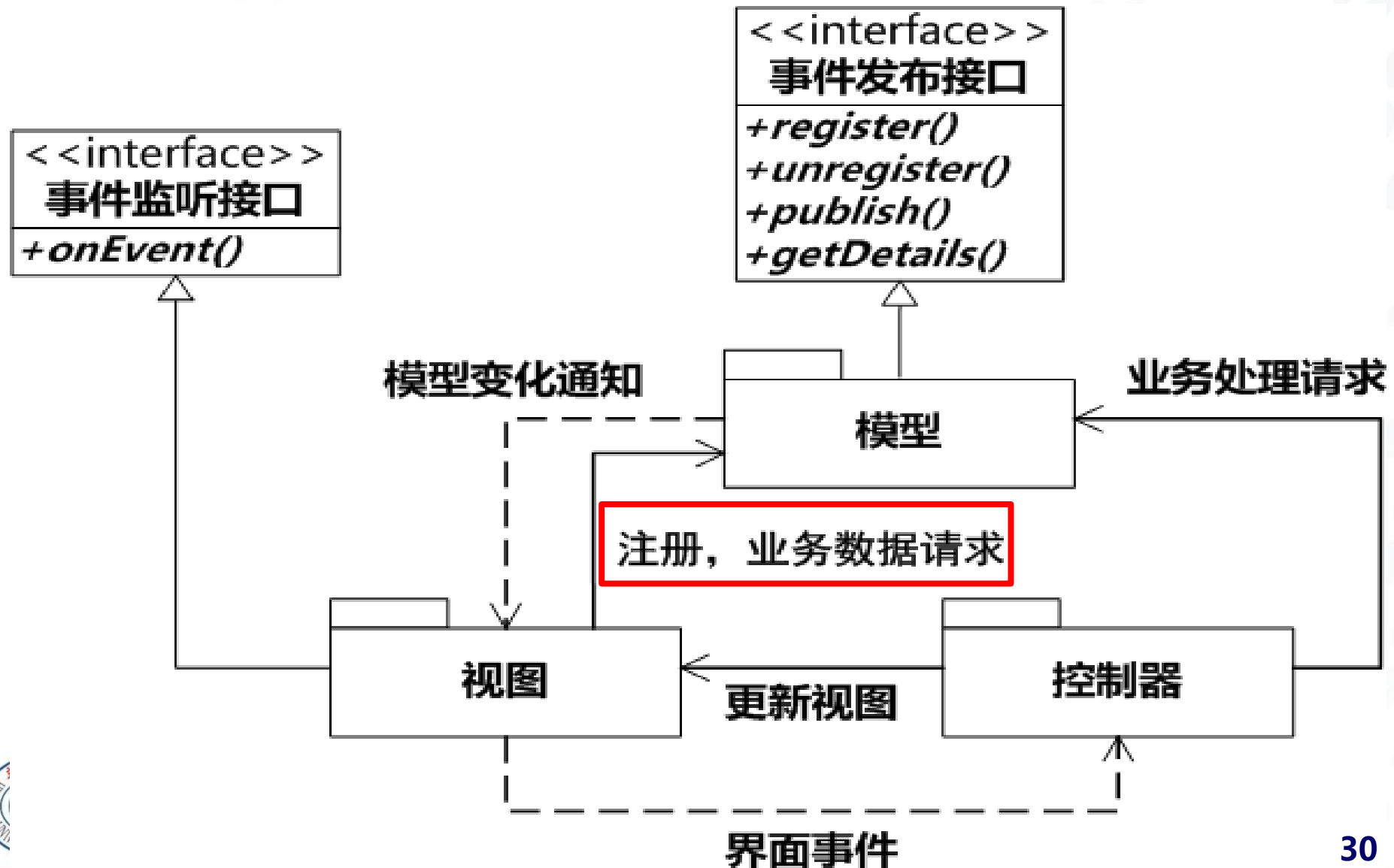
怎么办？



发布—订阅模式：拉(pull)模式



MVC模式与发布—订阅模式的结合



总结

1.

• MVC模式

2.

• 发布—订阅模式

3.

• 微内核模式

4.

• 反射模式

且听下回分解

